Townlight Core User Manual

Version: v1.2.1 (current downstream productization line)

Repository: https://github.com/townlight/core

License: Apache 2.0

> Formerly CivicCore / CivicSuite. The CivicSuite org and its repos moved to

> the townlight org on 2026-08-12; this package's import path is now

> townlight_core and its distribution name is townlight-core.

This manual has three audiences:

1. Non-technical evaluators - what Townlight Core is and why it matters.

2. IT and module developers - how to install it and consume the public API.

3. Architecture reviewers - what ships today, what is planned, and how the library fits into the Townlight stack.

---

1. Non-Technical Overview

What Townlight Core Is

Townlight Core is the shared platform library underneath Townlight. It is the common

Python package that Townlight modules use for migrations, LLM plumbing,

provenance metadata, audit-chain primitives, export manifests, and local city

configuration.

It is not an app a clerk or resident logs into. End users interact with

module applications such as CivicRecords AI or CivicClerk. Townlight Core is the

shared foundation those applications import.

What the current development line ships

- townlight_core.migrations - migration runner, idempotent guards, and the shared

schema baseline.

- townlight_core.db - shared SQLAlchemy declarative Base.

- townlight_core.llm - provider registry, prompt templates, model registry,

context utilities, and structured-output helpers.

- townlight_core.audit - hash-chained audit primitives for tamper-evident local

event streams plus legacy-compatible persisted audit-log verification
helpers.
- townlight_core.provenance - source, citation, document, and provenance metadata
contracts.
- townlight_core.connectors - offline import/export manifest schemas,
local-first import helpers for supported agenda-platform payloads, and
storage-neutral live-sync retry/circuit-breaker primitives plus vendor
delta request planning and source-list status projection.
- townlight_core.testing - no-network mock-city proof contracts for supported
agenda vendors, municipal OIDC, and backup-retention/off-host readiness.
- townlight_core.exports - static export-bundle manifest and checksum helpers.
- townlight_core.city_profile - local city/deployment configuration models.
- townlight_core.auth - bearer-token role helpers, staff-key route gates, and
trusted-header config/source-boundary helpers for protected or mixed
public/staff FastAPI routes.
- townlight_core.verification - content-bound browser release-evidence helpers.
- townlight_core.search - deterministic text normalization, matching, and
reciprocal-rank-fusion helpers.
- townlight_core.notifications - notice deadline planning and publication
compliance helpers with actionable warning codes.
- townlight_core.onboarding - storage-neutral onboarding profile field order,
answer parsing, completion-state, and next-question helpers.
- townlight_core.scheduling - storage-neutral cron validation and next-run
helpers for module background jobs.
- townlight_core.ingest - shared discovery/fetch contracts, cited-source
validation helpers, and document ingestion for PDF, DOCX, XLSX, CSV, EML,
HTML, and text files with sentence-aware chunking, local Ollama embeddings,
and pgvector-backed documents / document_chunks storage.

What the current development line does not ship yet

The following namespaces remain planned extraction targets:

townlight_core.catalog, townlight_core.exemptions, and townlight_core.scaffold.

townlight_core.onboarding now ships shared profile interview helpers, but

full web onboarding flows and persistence orchestration are still not

shipped platform behavior.

Credential storage, vendor-specific network adapters, vendor write-back,

worker scheduler runtimes, notification delivery queues, and legal

determinations are also not shipped platform behaviors. Downstream modules must

not promote those behaviors as shipped Townlight Core capability.

Why Municipal Teams Should Care

- Sovereignty: Local-first defaults keep cities in control of their data and

infrastructure.

- Reuse without coupling: Each Townlight module depends on the same

versioned primitives rather than copying logic.

- Auditability: Shared contracts for provenance, export bundles, and audit

chains make compliance evidence more consistent across modules.

---

2. Technical Guide for IT and Module Developers

Install from a Release Wheel

Townlight Core is distributed as GitHub release artifacts, not PyPI packages:

```
pip install https://github.com/townlight/core/releases/download/v1.2.1/civiccore-1.2.1-py3-none-any.whl
```

v1.2.1 predates this rename, so its wheel filename is still

civiccore-1.2.1-py3-none-any.whl (a real, already-published artifact). The

next release will ship as townlight_core-<version>-py3-none-any.whl under

the townlight-core distribution name.

Each release publishes SHA256SUMS.txt next to the wheel and source

distribution. Verify checksums before promoting a release artifact:

```
curl -L -o SHA256SUMS.txt \
  https://github.com/townlight/core/releases/download/v1.2.1/SHA256SUMS.txt
sha256sum -c SHA256SUMS.txt
```

v1.2.1 is the current published downstream productization line and includes

the shared document-ingestion pipeline used by the city-core release train.

v0.22.1 is the first

Townlight Core release with a Sigstore-signed

release-attestation.json and bundle. Earlier Townlight Core releases are retained

for historical installs only and must not be treated as provenance baselines

unless a future additive attestation is explicitly authorized, published, and

recorded in docs/ops/civiccore-tier1-retrofit-ledger.md.

For local development:

```
git clone https://github.com/townlight/core.git
cd core
pip install -e .[dev]
```

Use LLM Providers

```
from townlight_core.llm import get_provider

provider = get_provider("ollama", base_url="http://localhost:11434")
text = await provider.generate(
    system_prompt="You summarize municipal records.",
    user_content="Summarize this request: ...",
)
```

Ollama uses the base dependency set. OpenAI and Anthropic require their SDKs

only if those providers are instantiated:

```
pip install openai
pip install anthropic
```

Use Prompt Templates

```
from townlight_core.llm import render_template, resolve_template

template = await resolve_template(
    session,
    template_name="extract_request_fields",
    consumer_app="records-ai",
)
rendered = render_template(template, {"document_text": document_text})
```

The resolver checks app DB overrides first, code-level overrides second, and

Townlight Core defaults third. Missing variables produce actionable render errors.

Use Audit, Provenance, Manifest, Export, and City Profile Primitives

```
from townlight_core import (
    AuditActor,
    AuditHashChain,
    AuditSubject,
    CityProfile,
    ExportBundle,
    ImportManifest,
    PersistedAuditLogEntry,
    SourceReference,
```

```
    compute_persisted_audit_hash,
    validate_bundle,
    validate_manifest,
    verify_persisted_audit_chain,
)

chain = AuditHashChain()
chain.record_event(
    actor=AuditActor(actor_id="clerk-1", actor_type="staff"),
    action="packet_exported",
    subject=AuditSubject(subject_id="meeting-42", subject_type="meeting"),
    source_module="civicclerk",
)
assert chain.verify()

entry_hash = compute_persisted_audit_hash(
    previous_hash="0" * 64,
    timestamp="2026-05-01T12:00:00+00:00",
    actor_id="records-admin",
    action="request_created",
    details={"request_id": "RR-1001"},
)
assert verify_persisted_audit_chain([
    PersistedAuditLogEntry(
        previous_hash="0" * 64,
        entry_hash=entry_hash,
        timestamp="2026-05-01T12:00:00+00:00",
        actor_id="records-admin",
        action="request_created",
        details={"request_id": "RR-1001"},
    )
])[0]
```

These primitives are storage-neutral. They give downstream modules a consistent contract without dictating where records are stored.

Use Live Connector Sync Primitives

Use townlight_core.connectors for the shared retry and circuit-breaker state machine when a module pulls from a live vendor system. The module still owns its scheduler, tables, credentials, and vendor-specific fetch adapter.

```
from townlight_core.connectors import (
    SyncCircuitState,
    SyncRunResult,
    apply_sync_run_result,
    build_sync_operator_status,
    build_sync_source_status,
)

state = SyncCircuitState(connector="granicus", source_name="Granicus production")
state = apply_sync_run_result(
    state,
    SyncRunResult(records_discovered=4, records_succeeded=3, records_failed=1),
)
status = build_sync_operator_status(state)
print(status.public_dict()["message"])
source_status = build_sync_source_status(state, sync_schedule="0 2 * * *")
print(source_status.public_dict()["next_sync_at"])
```

Default behavior matches the CivicRecords AI pattern: healthy after successful

runs, degraded while failures remain, and circuit-open after five consecutive full-run failures. Grace-period sources open after two full-run failures so operators see the problem before scheduled pulls keep compounding it.
Use build_sync_source_status() for list/card views that need the same health decision plus active failure counts, pause state, last status, actionable fix copy, and next scheduled run.

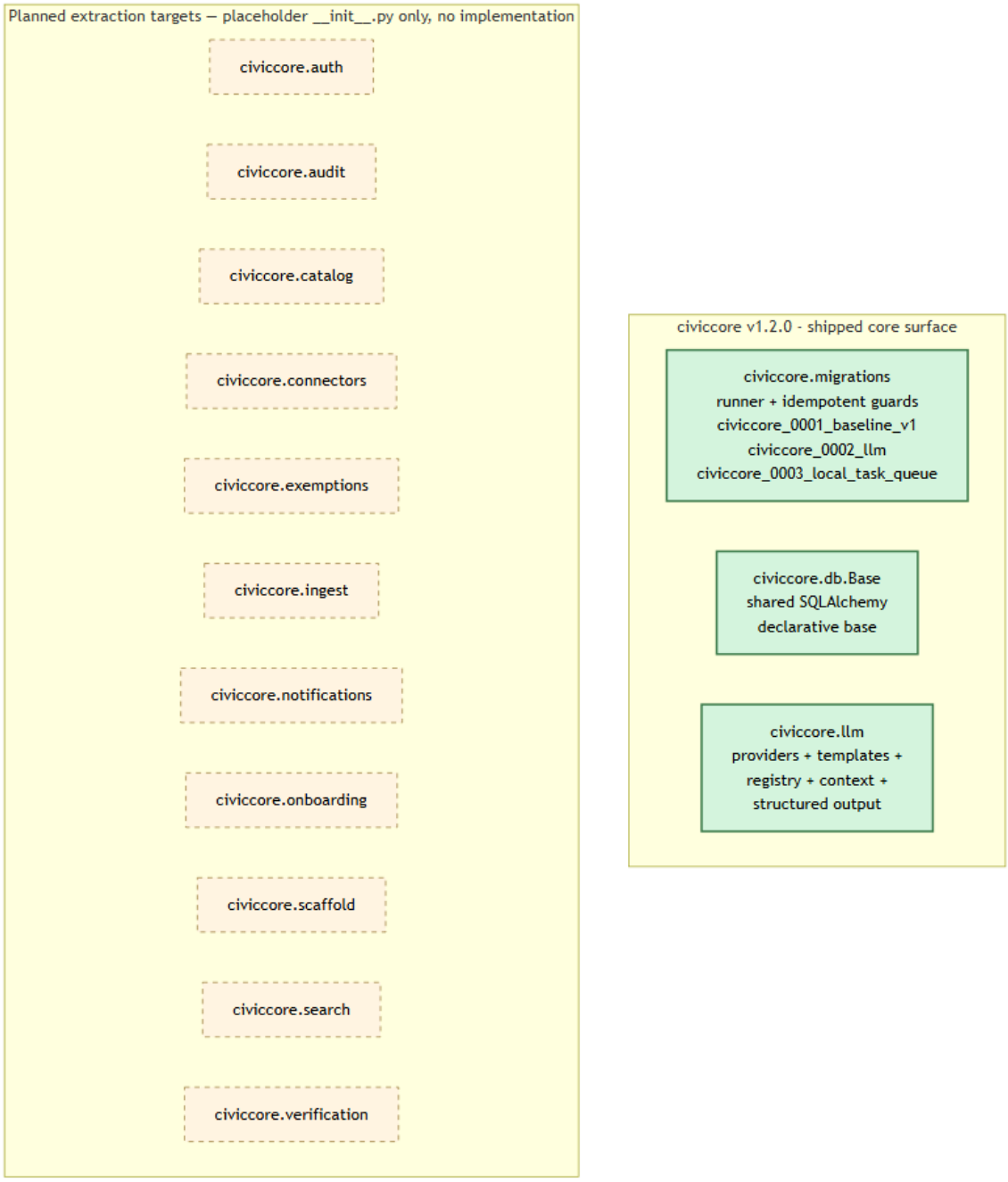
Run Migrations from a Consumer

Townlight Core migrations run before a downstream module's migrations. Consumer modules use Townlight Core's migration runner and keep their own version table so revision names do not collide.
The release gate verifies the Townlight Core migration chain, including civiccore_0001_baseline_v1 and civiccore_0002_llm.

---

3. Architecture Reference

Shipped vs Planned



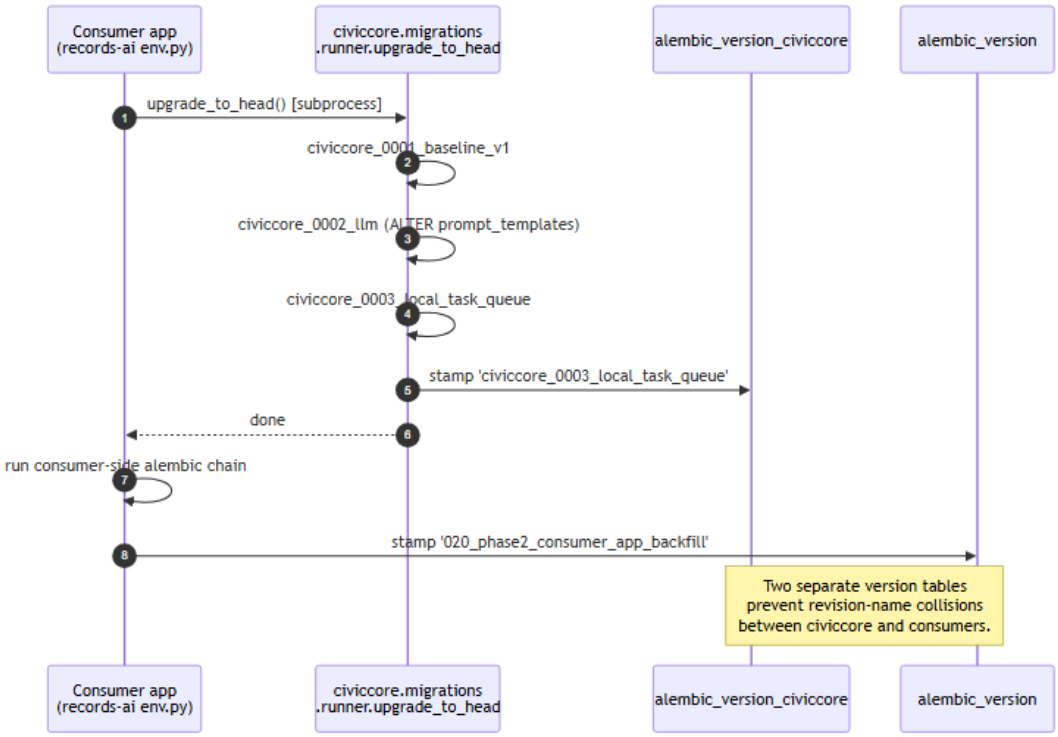
Shipped implementation in the current development line:

```
townlight_core/
  audit/        hash-chained audit primitives and persisted audit-log helpers
  city_profile/ local city/deployment configuration models
  connectors/   offline manifests, local-first import helpers, live-sync primitives
  db/           shared SQLAlchemy declarative Base
  exports/      static export-bundle helpers
  ingest/       shared contracts, cited-source validation, document ingestion,
                sentence-aware chunking, and pgvector embedding storage
  llm/          providers, templates, registry, context, structured output
  migrations/   migration runner and shared schema baseline
  notifications/ notice deadline + compliance helpers
  onboarding/   shared onboarding profile field-order/progress helpers
  scheduling/   cron validation and next-run helpers
  provenance/   source/citation/provenance metadata contracts
```

Still planned namespaces:

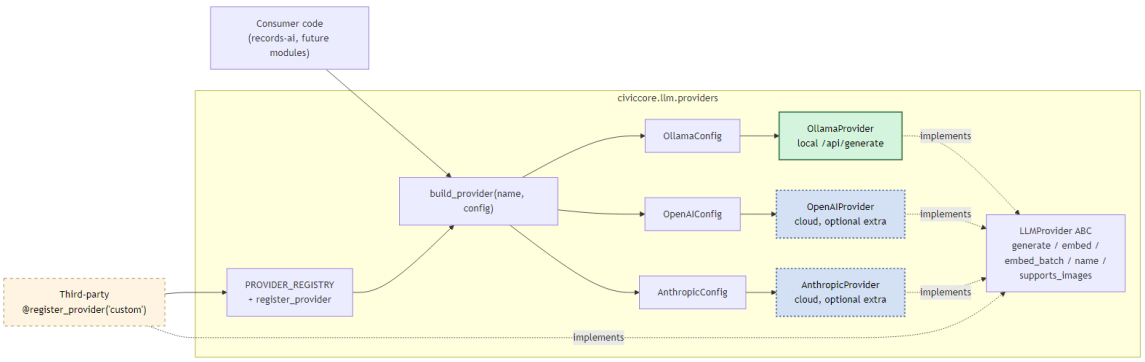
```
townlight_core/
  catalog/       future catalog primitives
  exemptions/    future 50-state public-records exemption engine
  notifications/ delivery queues and outbound orchestration remain future work
  onboarding/    future web onboarding UI/persistence flows
  scheduling/    scheduler runtime and task queue remain module-owned
  scaffold/      future scaffolding helpers
  verification/  future sovereignty verification
```

Migration Order



Consumer applications run Townlight Core migrations first, then their own module migrations. Separate Alembic version tables prevent revision-name collisions.

LLM Provider Abstraction



The provider abstraction keeps local Ollama as the sovereignty-first default

while allowing explicitly configured cloud providers where a city authorizes

them.

Compatibility

Current v0.1.0 module foundations still pin older civiccore lines.

Production-depth consumers should move only to the released Townlight Core version

recorded in their compatibility matrix.

The suite-wide matrix lives at:

https://github.com/townlight/townlight/tree/main/docs/compatibility

---

Appendix: Where to File Issues

- Townlight Core bug: https://github.com/townlight/core/issues

- Suite-wide design issue: https://github.com/townlight/townlight/issues

- Security issue: follow SECURITY.md; do not file publicly.

The decision tree in CONTRIBUTING.md has the full routing rules.